

C++ Template 延伸自主練習作業報告

主題： **STL** 與基礎資料結構入門

系級： 資訊工程學系 三年級

學生姓名： 林明亮

學號： S11259053

GitHub Repository: <https://github.com/linliangliang61-lgtm/cpp-student-system>

1. 自主學習主題與目標

本次專題的學習目標在於將 Chapter 16 所學的 Template（模板）函式與類別觀念，延伸應用至 C++ 標準模板庫 (STL)。透過實作「學生成績管理系統」，驗證自己對於抽象資料結構的掌握度。不僅要了解如何為不同的資料結構（如動態陣列、樹狀結構）選擇最適合的 STL 容器，也期許透過這次從無到有的系統建置，為未來面對資工所資料結構與演算法的進階考題打下紮實的底層實作基礎。

2. Template 與 STL 的關係

C++ 的 Template 是泛型編程 (Generic Programming) 的核心基石，它允許開發者編寫與資料型態無關的邏輯程式碼。而 STL (Standard Template Library) 本質上就是 Template 概念的極致實踐。STL 將資料容器 (Containers) 與對資料的操作演算法 (Algorithms) 巧妙地分開，透過 Template 的機制，我們可以讓底層邏輯輕易裝載 `int`、`string`，甚至是自定義的 `struct Student` 物件，大幅提升程式碼的重用性。

3. 常見 STL 容器用途整理

在本次自主學習中，我梳理了以下常見容器的底層概念與最佳使用時機：

- **vector**: 具備動態擴展能力的陣列。適合需要頻繁且快速循序存取 ($O(1)$) 的場景，但在陣列中間插入或刪除元素的成本較高 ($O(N)$)。
- **map**: 基於紅黑樹 (Red-Black Tree) 實作的鍵值對 (Key-Value) 容器。適合需要以特定 Key (如學號) 進行快速查找、新增、刪除的場景，其時間複雜度皆為高效率的 $O(\log N)$ 。
- **stack / queue**: 分別代表後進先出 (LIFO) 與先進先出 (FIFO) 的配接器容器，適合用來處理需要嚴格順序控制的邏輯，例如括號匹配檢驗或服務排隊模擬。
-

4. 第 2 週小專題設計說明

在「學生成績管理系統」的設計中，考量到系統效能與防呆機制，我選擇揉合不同的容器優勢：首先，考量到系統需要「防止學號重複新增」與「高頻率的學號查詢」，我選擇以 `map<string, Student>` 作為系統的主記憶體結構。將學號字串作為 `Key`，`Student` 結構作為 `Value`。這樣在新增時能以 $O(\log N)$ 的極快速度判定學號是否重複，遠勝於使用 `vector` 搭配迴圈的 $O(N)$ 循序檢查。然而，當使用者呼叫「依成績排序」功能時，由於 `map` 預設只能嚴格根據 `Key`（學號）進行內部紅黑樹排序，我設計了一個中繼轉換機制：將 `map` 內的元素提取至 `vector<Student>` 中，再利用 `<algorithm>` 中的 `std::sort` 完成成績降冪排序，完美結合兩者的優點。

5. 主要程式架構與重要程式片段

(1) 固定挑戰題：**Template Function** 的實作與呼叫 設計通用的 `getMax` 與 `getMin`，並在成績統計時明確指定型別呼叫，精準抓出全班最高與最低分。

C++

```
template <class T>
```

```
T getMax(T a, T b) { return (a > b) ? a : b; }
```

```
// 在迴圈中統計成績極值
```

```
max_score = getMax<int>(max_score, current_score);
```

```
min_score = getMin<int>(min_score, current_score);
```

(2) 結合 **Lambda 運算式**與 `std::sort` 的排序演算法 透過 C++11 的 `Lambda`

語法，自訂排序規則，讓 `vector` 內的資料依照成績高低排列。

C++

```
sort(sorted_students.begin(), sorted_students.end(), [](const Student& a, const
Student& b) {

    return a.score > b.score;

});
```

6. 執行畫面或輸出結果

```
[新增學生資料]
請輸入學號: S001
請輸入姓名: Alice
請輸入成績 (0-100): 85
[成功] 學生 Alice 資料新增完成！

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S002
請輸入姓名: Bob
請輸入成績 (0-100): 42
[成功] 學生 Bob 資料新增完成！

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S003
請輸入姓名: Charlie
請輸入成績 (0-100): 95
[成功] 學生 Charlie 資料新增完成！

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S001
[錯誤] 學號 S001 已存在，拒絕新增。

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S004
請輸入姓名: David
請輸入成績 (0-100): 150
[錯誤] 成績必須介於 0 到 100 之間。

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
```

圖一：測試新增資料功能。系統成功紀錄了 Alice (85 分)、Bob (42 分)、Charlie (95 分) 的資料。

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S001
請輸入姓名: Alice
請輸入成績 (0-100): 85
[成功] 學生 Alice 資料新增完成！
```

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S002
請輸入姓名: Bob
請輸入成績 (0-100): 42
[成功] 學生 Bob 資料新增完成！
```

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S003
請輸入姓名: Charlie
請輸入成績 (0-100): 95
[成功] 學生 Charlie 資料新增完成！
```

圖二：測試系統防呆機制。當輸入重複的學號 S001，以及超過範圍的 150 分時，系統成功觸發錯誤攔截，拒絕寫入資料，確保資料庫的一致性。

```
=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項: 1

[新增學生資料]
請輸入學號: S001
[錯誤] 學號 S001 已存在，拒絕新增。
```

=== 學生成績管理系統 ===

1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

=====

請輸入選項: 1

[新增學生資料]

請輸入學號: S004

請輸入姓名: David

請輸入成績 (0-100): 150

[錯誤] 成績必須介於 0 到 100 之間。

圖三：測試列出所有學生。系統能完整走訪 map 容器，並透過 <iomanip> 函式庫整齊對齊輸出格式。

```
=====
請輸入選項： 2

[所有學生列表]
學號      姓名      成績
-----
S001      Alice      85
S002      Bob        42
S003      Charlie     95

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項： █
```

圖四：測試依成績排序功能。成功將資料轉交由 vector 儲存，並使用 std::sort 完成成績由高到低（Charlie -> Alice -> Bob）的降冪排序。

```
=====
請輸入選項： 3

[依成績排序 (高到低)]
學號      姓名      成績
-----
S003      Charlie     95
S001      Alice       85
S002      Bob         42

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項： █
```

圖五：測試查詢學生功能。運用 `map::find` 的 $O(\log N)$ 搜尋特性，精準查出存在的學生 (Bob)，並正確回報查無此學號 (S999) 的狀況。

```
=====
請輸入選項：4

[查詢學生]
請輸入要查詢的學號：S002
[找到資料]
學號：S002
姓名：Bob
成績：42

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項：█
```

```
=====
請輸入選項：4

[查詢學生]
請輸入要查詢的學號：S999
[錯誤] 查無此學號。

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit
=====
請輸入選項：█
```

圖六：測試成績統計與 **Template** 挑戰題。系統正確計算出全班總人數、平均、及格與不及格人數，且 **Template Function** 精準抓出最高分 95 與最低分 42。

```
=====
請輸入選項：5

[成績統計]
總人數：3
全班平均：74.00
最高分：95
最低分：42
及格人數：2
不及格數：1

=== 學生成績管理系統 ===
1. Add student
2. List all students
3. Sort by score
4. Search by id
5. Show statistics
0. Exit

=====
請輸入選項：█
```

7. GitHub repo link 與檔案結構說明

- **Repository Link:** <https://github.com/linliangliang61-lgtm/cpp-student-system>
- 檔案結構:
 - main.cpp: 系統主程式碼，包含所有功能實作與 **Template** 應用。
 - README.md: 專案簡介與編譯環境說明。

- report.pdf: 延伸自主練習作業結案報告。

8. 遇到的問題與解決方式

問題：Windows PowerShell 終端機亂碼問題 在撰寫與編譯程式初期，由於我的 main.cpp 採用了國際標準的 UTF-8 編碼儲存，但在 Windows 環境下使用 g++ 編譯並執行時，PowerShell 預設的 Big5 (950) 編碼無法正確解析 UTF-8 的中文字元，導致選單與提示訊息全部變成亂碼（Mojibake）。**解決方式：調整編譯器參數** 為了解決這個問題，我深入研究了編譯器的參數設定，最終在執行 g++ 編譯時加入了 -fexec-charset=BIG5 參數（指令為 g++ -std=c++17 -fexec-charset=BIG5 main.cpp -o main）。這個做法能在維持原始碼檔案為純淨 UTF-8 的前提下，強制將輸出執行檔的字元集轉換為 Windows 相容的格式，成功且優雅地解決了亂碼問題，且上傳至 GitHub 的程式碼也不會出現亂碼。

9. 學習心得與未來可延伸方向

本次作業讓我深刻體會到「沒有絕對最好的資料結構，只有最適合當下情境的資料結構」。透過揉合 map 的查詢優勢與 vector 的排序彈性，系統的健壯度大幅提升。這也是我第一次完整實踐 C++17 標準與 STL 的整合應用，這些底層容器的理解對我未來準備進階考試有著直接的幫助。

未來延伸：目前的系統資料在程式關閉後就會消失。未來若有機會，我希望能引入 C++ 的 <fstream> 檔案讀寫功能，將成績清單序列化儲存成 .csv 或 .txt 檔案，讓這支程式成為真正具備持久化存取能力的系統。